IMT Nord Europe
École Mines-Télécom
IMT-Université de Lille

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

Université Catholique de Lille 1875

LITL

Programmation Orientée Objet (POO) à destination de la robotique (2)

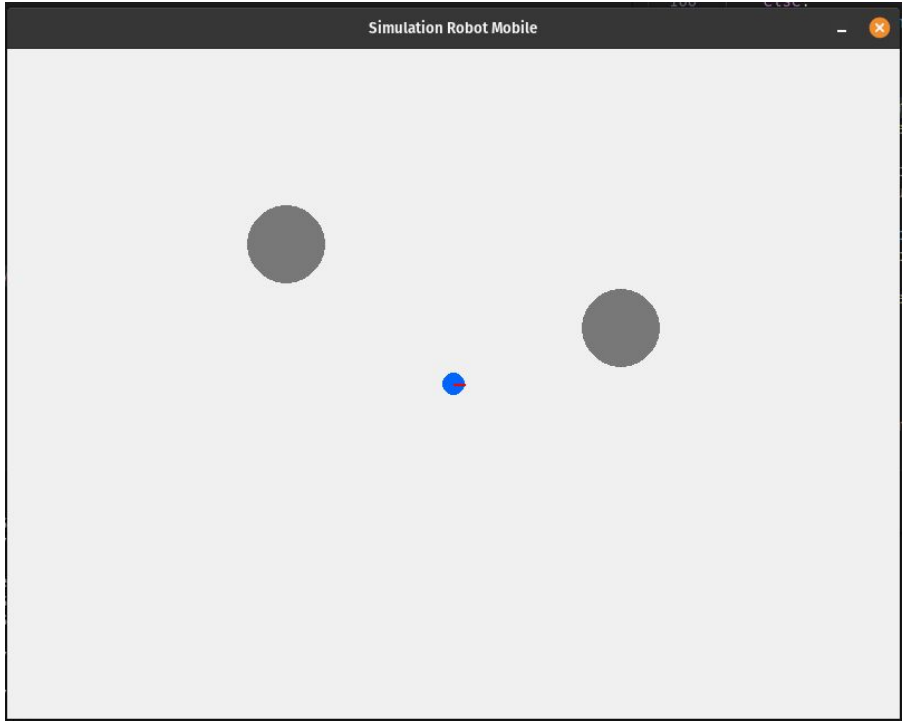
Adam GOUGUET
adam.gouguet@imt-nord-europe.fr

2025-2026

Introduction : Projet fil rouge simulation robotique mobile

Durant les premiers TPs vous avez :

- Programmer un robot mobile se déplacant dans un environment bidimensionnelle.
- Fait une simulation à partir de Pygame.
- Modéliser cela avec le design patern MVC.

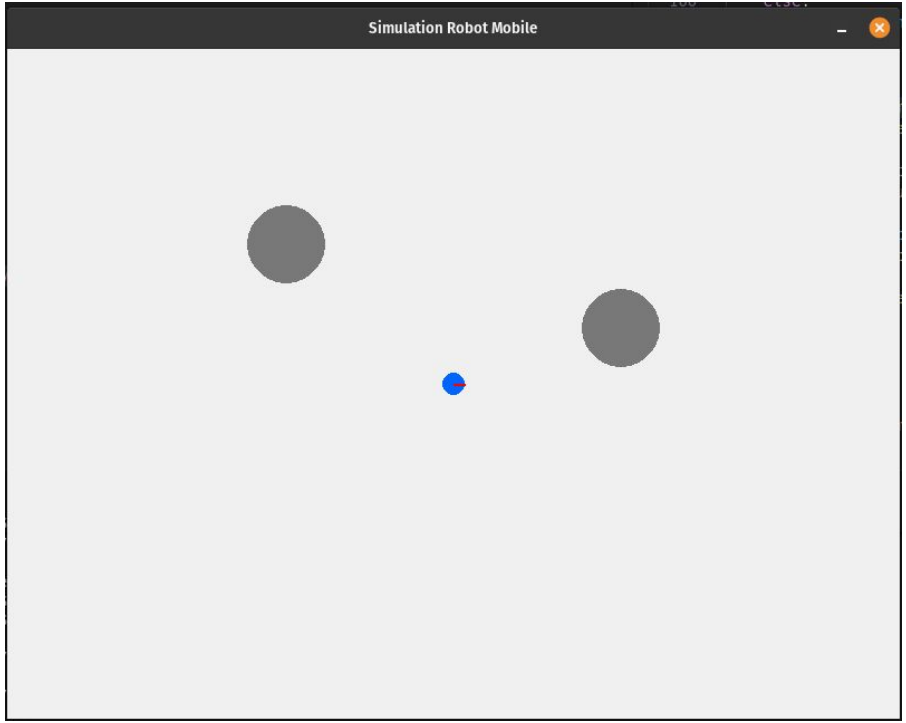


Introduction : Projet fil rouge simulation robotique mobile

L'objectif maintenant est d'ajouter des fonctionalités propres à votre simulation robotique. Imaginer un scénario/un but à votre application :

- transport de paquets.
- course de robot.
- soccer bot.
- robot secouriste.
- ...

On se base sur le scénario de transport de marchandise pour la suite. Je vous propose une liste non exhaustive des fonctionalités que vous pouvez implémenté **avec à chaque fois une partie cours (robotique) que vous devez connaitre.**



Notation : Projet fil rouge simulation robotique mobile

Critère	Note
Fonctionnalités implémentées (quantité et difficulté)	/10
Présentation orale et démonstration	/4
Évaluation par les pairs	/2
Gestion du projet et répartition du travail au sein du groupe	/2
Documentation (README, explications techniques)	/1
Qualité du code (lisibilité, modularité, bonnes pratiques)	/1

IMT Nord Europe
École Mines-Télécom
IMT-Université de Lille

Université Catholique de Lille 1875

LITL

Fonctionalités avancées

Packaging, configuration, logs, arguments

Fonctionnalités Python à implémenter :

- **Packaging** : structure modulaire, distribution, installation simplifiée
- **Configuration** : fichiers config (YAML/JSON), paramètres dynamiques
- **Logs** : suivi d'exécution, debug, traçabilité des événements (module logging)
- **Arguments CLI** : flexibilité d'exécution, personnalisation des runs (module parsearg)

Objectif : *Code robuste, maintenable et réutilisable*

Capteurs

Lidar / Scanner laser

- Mesure les distances aux obstacles autour du robot.
- Donne une vision 2D ou 3D de l'environnement.
- Détection d'obstacles, cartographie, navigation autonome.

Encodeurs de roues

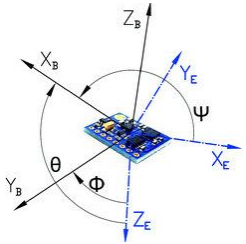
- Mesurent la rotation des roues pour calculer la distance parcourue.
- Utile pour l'odométrie, suivi de trajectoire et correction de position.

GPS / GNSS

- Fournit la position globale (latitude, longitude, altitude).
- Utile pour les robots en extérieur ou véhicules autonomes.
- Limité en précision pour les environnements fermés.

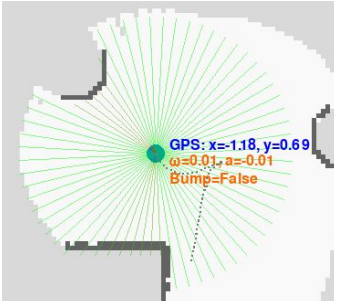
Capteurs de contact / Bumper

- Détectent collision ou contact physique avec l'environnement.
- Utilité : sécurité, évitement immédiat d'obstacles.



IMU (Inertial Measurement Unit)

- Mesure accélérations et vitesses angulaires.
- Estimation de la position via odométrie inertielle, stabilisation.



Simulation Réaliste

Pourquoi un modèle moteur réaliste ?

- **Modèle simple** : réponse instantanée = irréaliste physiquement
- **Problème** : pas d'inertie, pas de limites, pas de perturbations
- **Conséquence** : algorithmes validés en simulation mais échouent sur robot réel

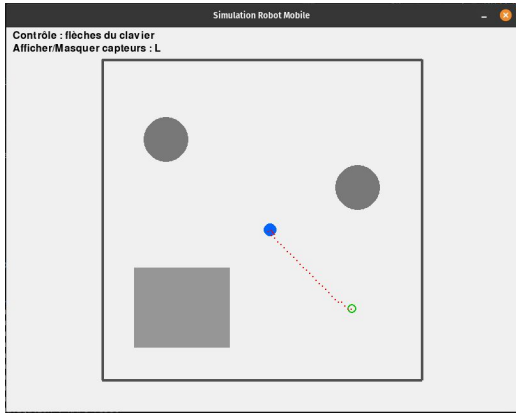
Améliorations d'un modèle réaliste

- **Inertie** : accélération limitée (pas de changement instantané)
- **Saturation** : vitesses max physiques respectées
- **Frottements** : ralentissement naturel
- **Bruit** : capteurs / actionneurs imparfaits
- **Puissance** : dépend de la masse du robot

Navigation Autonome Facile : Navigation réactive

Navigation autonome - Approche Réactive

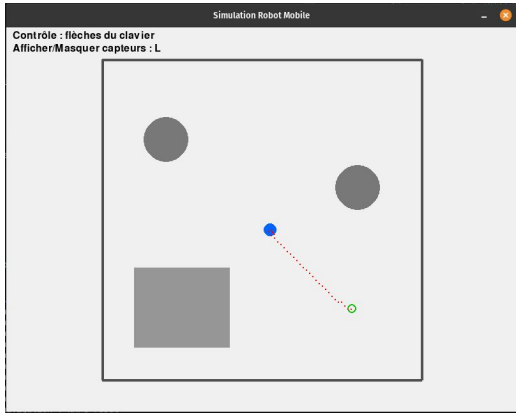
- **Principe** : décisions instantanées basées uniquement sur les capteurs
- **Logique** : règles simples (*if obstacle -> tourner*)
- **Mémoire** : aucune connaissance globale de l'environnement
- **Temps réel** : réaction rapide, calcul léger

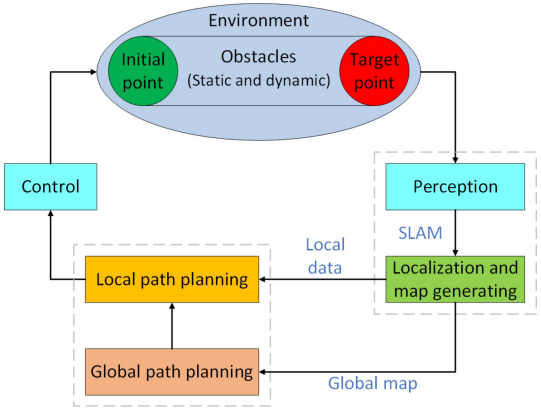


Navigation Autonome

Navigation autonome (robot mobile) :

- **Percevoir** : capteurs -> obstacles, carte, localisation
- **Comprendre** : fusion des données -> modèle du monde
- **Planifier le chemin** :
  a. **Planifier globalement** : *planificateur global* -> trajectoire optimale vers l'objectif (A*)
  b. **Planifier localement** : *planificateur local / contrôleur* -> évitement d'obstacles, ajustements temps réel et commandes moteurs précises (ex : Dynamic Window Approach / Model Predictive Control)
- **Corriger** : feedback capteurs -> ajustement continu





https://www.nature.com/articles/s41598-024-72857-3

Navigation Autonome : Reinforcement Learning

Navigation autonome par Reinforcement Learning

- **Principe** : le robot apprend seul par essai-erreur via récompenses/pénalités
- **Entrées** : observations capteurs (distance obstacles, position, carte partielle)
- **Actions** : commandes moteurs (vitesse, rotation)

Objectif : maximiser la récompense cumulée (atteindre but, éviter collisions)

Boucle d'apprentissage

- Observer l'état
- Choisir une action (politique)
- Recevoir une récompense
- Mettre à jour le modèle

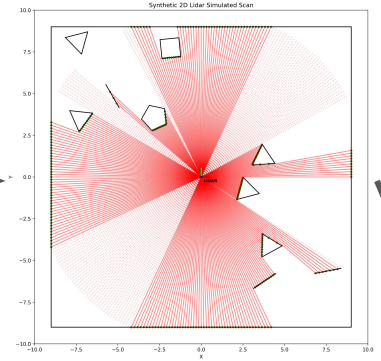
Cycle répété des milliers (millions) de fois -> comportement optimal

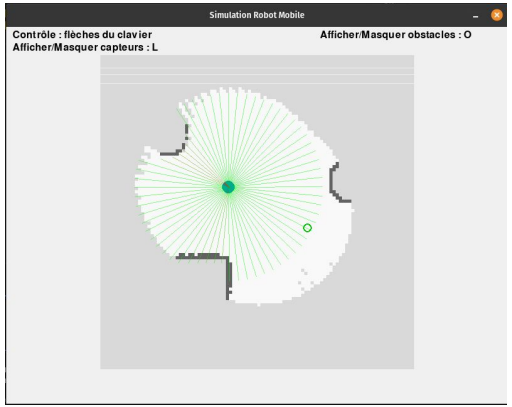
Quels est la meilleur approche pour faire de la navigation autonome ?

(Vous pouvez faire un comparatif avancée avec métriques : distance, temps, rotation, etc...)

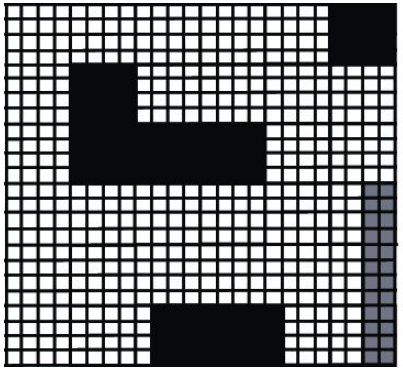
Cartographie

LiDAR
(light detection and ranging)

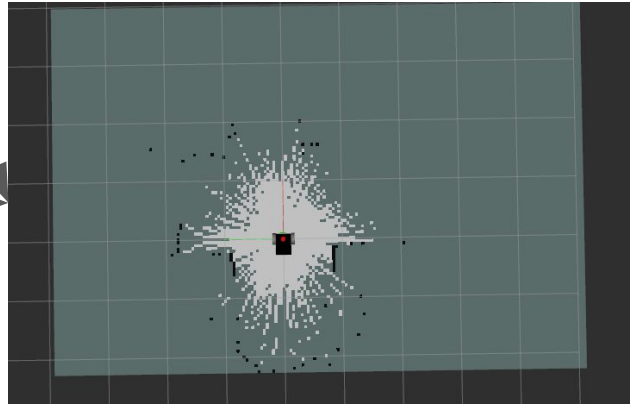




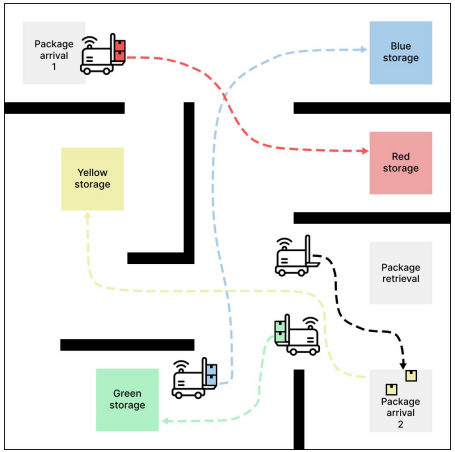
Grille d'occupance



Free area
Occupied area
Unknown area



Scenario



Allocation de tâches pour du transport autonome

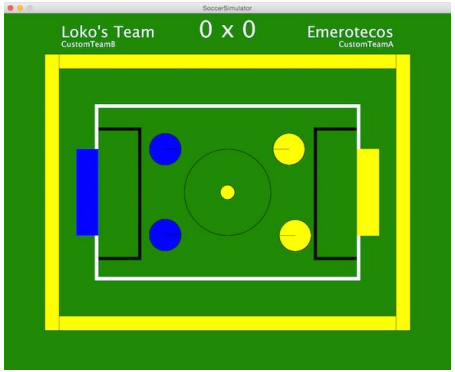
- Modélisation du fonctionnement d'une usine (arriver des paquets, ajouter/supprimer robot, etc…)
- Coordination d'une flotte de robot et allocation de tâches.
- Définir les zones de dépôt et d'arriver de paquets.

Mission de secours

- Environnement dynamique -> carte change en cours d'execution (apparition de feu par exemple)
- Cartographie et annoter les personnes en danger.
- Modéliser des agents dynamique (humains) dans l'environnement.

Patrouille

- Robot suit un parcours ou patrouille une zone.
- Suivi de chemin, détection d'événements, retour automatique à la base.



Soccer Bot

- Coordination d'une flotte de robot.
- Plusieurs roles (attaquant, défenseur, gardien).
- Voir les algorithmes Boids.
- Faire s'affronter 2 IA pour tester la meilleur.

Course d'obstacles

- Robot traverse un parcours complexe avec obstacles fixes et mobiles.
- Contrôle réactif, planification locale, utilisation efficace des capteurs.